Practical - 8

Aim:

Implement the object oriented programming concepts using Python.

Q1: Create a class Employee with data members: name, department and salary. Use constructor to initialize values and display() method for printing information of three employees.

Code:

```
class Employee:
    def __init__(self, name, department, salary):
        self.name = name
        self.department = department
        self.salary = salary

    def display(self):
        print(f"Name:                          {self.name}\nDepartment:
{self.department}\nSalary: {self.salary}")

emp1 = Employee("Vatsal", "Engineering", 100000)
emp1.display()
```

Output:

```
Name: Vatsal
Department: Engineering
Salary: 100000
```

Q2: Write a program to create class Student with following attributes: instance variables enrollment_no, name and branch; instance methods get_value() and print_value(); class variable cnt; static method show(). Variable cnt counts number of instances created and show() method displays value of cnt.

Code:

```
class Student:
    count = 0  # class variable to keep track of the number of
students

    def __init__(self, enrollment_number, name, branch):
        self.enrollment_number = enrollment_number
        self.name = name
        self.branch = branch
        # count number of students innstance
        Student.count += 1


    def get_value(self):
        student = [
            self.enrollment_number,
            self.name,
            self.branch,
        ]
        return student

    def show_count(self):
        print("Total number of students:", Student.count)

student1 = Student("142", "Vatsal", "Computer Engineering")
student2 = Student("143", "Honey", "Information Technology")


print(student1.get_value())
```

```
print(student2.get_value())
student1.show_count()
```

Output:

```
['142', 'Vatsal', 'Computer Engineering']
['143', 'Honey', 'Information Technology']
Total number of students: 2
```

Q3: Write a program to overload ** (exponential) operator.

Code:

```
class Power:
    def __init__(self, base):
        self.base = base

    def __pow__(self, exponent):
        return self.base ** exponent

power1 = Power(2)
result = power1 ** 3
print("Result of 2 ** 3 with overloaded ** operator:", result)
```

Output:

```
Result of 2 ** 3 with overloaded ** operator: 8
```

Q4: Create class Hospital having attributes patient_no, patient_name and disease_name and an instance p1. Show use of methods getattr(), setattr(), delattr(), and hasattr() for p1. Display values of attributes __dict__, __doc__, __name__, __module__, __bases__ with respect to class Hospital. Delete instance p1 in the end.

Code:

```
class Hospital:
    def __init__(self, patient_no, patient_name, disease_name):
        self.patient_no = patient_no
        self.patient_name = patient_name
        self.disease_name = disease_name

    def getattr(self, attribute):
        return getattr(self, attribute)

    def setattr(self, attribute, value):
        setattr(self, attribute, value)

    def delattr(self, attribute):
        delattr(self, attribute)

    def hasattr(self, attribute):
        return hasattr(self, attribute)

p1 = Hospital(1, "Vatsal", "Flu")
print(p1.getattr('patient_name'))  # Output: Vatsal
p1.setattr('disease_name', 'Cold')
print(p1.getattr('disease_name'))  # Output: Cold
print(p1.hasattr('patient_no'))  # Output: True
p1.delattr('patient_no')
print(p1.hasattr('patient_no'))  # Output: False

print(p1.__dict__)
print(p1.__doc__)
print(p1.__class__.__name__)
```

```
print(p1.__module__)
print(p1.__class__.__bases__)
```

Output:

```
Vatsal
Cold
True
False
{'patient_name': 'Vatsal', 'disease_name': 'Cold'}
None
Hospital
__main__
(<class 'object'>,)
```

Q5: Design a class Lion having method roar() and a class Cub having method play() which inherits class Lion. Use instance of Cub called- simba to access methods roar() and play(). Define public attribute legs, protected attribute ears and private attribute mane of class Lion. Show accessibility of these variables according to their scope.

Code:

```
class Lion:
    def __init__(self):
        self.legs = 4
        self._ear = 2
        self.__mane = 1

    def roar(self):
        print("The lion roars loudly!")

class cub(Lion):
    def __init__(self):
        super().__init__()
    def play(self):
        print("The cub plays around!")

cub1 = cub()
cub1.roar()  # Inherited method from Lion
cub1.play()  # Method defined in cub class
print(cub1.legs)  # Accessing public attribute from Lion
print(cub1._ear)  # Accessing protected attribute from Lion
```

Output:

```
The lion roars loudly!
The cub plays around!
4
2
```

Code:

```
print(cub1.__mane)
```

Output:

```
---------------------------------------------------------------------------
AttributeError                            Traceback (most recent call last)
Cell In[34], line 1
----> 1 print(cub1.__mane)

AttributeError: 'cub' object has no attribute '__mane'
```

Q6: Class Person with attributes- name and age is inherited by class SportPerson with attribute sport_name. Use appropriate __init__() method for both classes. Call parent __init__() method from child __init__() method with the help of (A) super() method (B) parent class name.

Code:

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def display(self):
        print(f"Name: {self.name}, Age: {self.age}")

class SportPerson(Person):
    def __init__(self, name, age, sport):
        super().__init__(name, age)
        self.sport_name = sport
    def display(self):
        super().display()
        print(f"Sport: {self.sport_name}")

class SportPerson1(Person):
    def __init__(self, name, age, sport):
        Person.__init__(self, name, age)
        self.sport_name = sport
    def display(self):
        Person.display(self)
        print(f"Sport: {self.sport_name}")

sportsperson1 = SportPerson("Vatsal", 21, "Football")
sportsperson1.display()

sportsperson2 = SportPerson1("Honey", 22, "Badminton")
sportsperson2.display()
```

Output:

```
Name: Vatsal, Age: 21
Sport: Football
Name: Honey, Age: 22
Sport: Badminton
```

Q7: Write programs to implement following scenarios where A, B, C, D, E and F are classes and check() is a method. In both scenarios, which check() method is called, when we execute statement- E().check()

Code1:

```
class A:
    def check(self):
        print("This is class A")

class B(A):
    pass

class C(B):
    def check(self):
        print("This is class C")

class D(A):
    def check(self):
        print("This is class D")

class E(B, D):
    pass

e1 = E()
e1.check()
```

Output1:

```
This is class D
```

Code2:

```
class A:
    def check(self):
        print("This is class A")

class B(A):
    pass

class F:
    def check(self):
        print("This is class F")

class C(F):
    def check(self):
        print("This is class C")

class D(F):
    def check(self):
        print("This is class D")

class E(B, C, D):
    pass

e2 = E()
e2.check()
```

Output2:

```
This is class A
```

Q8: Write a program in which Python and Snake sub classes implement abstract methods- crawl() and sting() of the super class Reptile. What is the output of following statements?
i) **issubclass**(Python, Reptile) ii) **isinstance**(Snake(), Reptile)

Code:

```
from abc import ABC, abstractmethod

class Reptile(ABC):
    @abstractmethod
    def sting(self):
        pass

    @abstractmethod
    def crawl(self):
        pass

class Snake(Reptile):
    def sting(self):
        print(f"{self.name} can sting with its venom.")
    def crawl(self):
        print(f"{self.name} can crawl on the ground.")

class Python(Snake):
    def sting(self):
        print(f"{self.name} does not have venom but can constrict
its prey.")
    def crawl(self):
        print(f"{self.name} can crawl on the ground like other
snakes.")

print(issubclass(Python, Reptile))  # True
print(isinstance(Python(), Reptile))  # True
```

Output:

```
True
True
```

Practice Exercise:

1. Create a class 'BankAccount' with:

- Private attributes: __balance, __account_number
- Constructor to initialize values
- Methods: deposit(), withdraw(), get_balance()
- Class variable to track total_accounts
- Static method to display bank statistics
- Demonstrate encapsulation and static methods

Code:

```
class BankAccount:
    __count = 0
    def __init__(self, account_number, account_holder, balance):
        self.account_number = account_number
        self.account_holder = account_holder
        self.balance = balance
        BankAccount.__count += 1
        print(f"Account created for {self.account_holder} with
balance {self.balance}")

    def deposit(self, amount):
        self.balance += amount
        print(f"Deposited {amount} in {self.account_holder}'s
account. New balance: {self.balance}")

    def withdraw(self, amount):
        if amount > self.balance:
            print(f"Insufficient funds in {self.account_holder}'s
account")
        else:
            self.balance -= amount
            print(f"Withdrew {amount} from {self.account_holder}'s
account. New balance: {self.balance}")

    def get_balance(self):
```

```
        return self.balance

    @staticmethod
    def get_statistics():
        print(f"Total number of accounts: {BankAccount.__count}")

account1 = BankAccount("123456", "Vatsal", 1000)
account1.deposit(500)
account1.withdraw(200)
print("Current balance of Vatsal:", account1.get_balance())
BankAccount.get_statistics()
print("---------------------------------------")
account2 = BankAccount("654321", "Honey", 2000)
account2.deposit(1000)
account2.withdraw(500)
print("Current balance of Honey:", account2.get_balance())
BankAccount.get_statistics()
```

Output:

```
Account created for Vatsal with balance 1000
Deposited 500 in Vatsal's account. New balance: 1500
Withdrew 200 from Vatsal's account. New balance: 1300
Current balance of Vatsal: 1300
Total number of accounts: 1
---------------------------------------
Account created for Honey with balance 2000
Deposited 1000 in Honey's account. New balance: 3000
Withdrew 500 from Honey's account. New balance: 2500
Current balance of Honey: 2500
Total number of accounts: 2
```

2. Design a class hierarchy for a Vehicle System:

- Base class Vehicle with brand, model, year
- Two child classes: Car(seats, fuel_type) and Bike(engine_cc)
- Override str method in all classes
- Implement method_overloading for calculate_price()
- Use super() to access parent methods
- Show inheritance and polymorphism

Code:

```
class Vehicle:
    def __init__(self, brand, model, year):
        self.brand = brand
        self.model = model
        self.year = year

    def __str__(self):
        return f"{self.year} {self.brand} {self.model}"

    def calculate_price(self):
        base_price = 20000
        age = 2026 - self.year
        depreciation = age * 1000
        return max(base_price - depreciation, 0)

class Car(Vehicle):
    def __init__(self, brand, model, year, seats, fuel_type):
        super().__init__(brand, model, year)
        self.seats = seats
        self.fuel_type = fuel_type

    def __str__(self):
        return super().__str__() + f", Seats: {self.seats}, Fuel
Type: {self.fuel_type}"

    def calculate_price(self):
```

```
        base_price = super().calculate_price()
        if self.fuel_type == "Electric":
            return base_price + 5000  # Electric cars have a
premium
        return base_price


class Bike(Vehicle):
    def __init__(self, brand, model, year, engine_cc):
        super().__init__(brand, model, year)
        self.engine_cc = engine_cc

    def __str__(self):
        return super().__str__() + f", Engine CC: {self.engine_cc}"

    def calculate_price(self):
        base_price = super().calculate_price()
        if self.engine_cc > 1000:
            return base_price + 2000  # High engine capacity bikes
have a premium
        return base_price


car1 = Car("Tesla", "Model S", 2020, 5, "Electric")
bike1 = Bike("Yamaha", "R1", 2018, 998)
print(car1)
print("Price of the car:", car1.calculate_price())
print(bike1)
print("Price of the bike:", bike1.calculate_price())
```

Output:

```
2020 Tesla Model S, Seats: 5, Fuel Type: Electric
Price of the car: 19000
2018 Yamaha R1, Engine CC: 998
Price of the bike: 12000
```

3. Create a University Management System:

- Base class Person(name, age)
- Child classes: Student(roll_no, course) and Faculty(department, salary)
- Use property decorators for getters/setters
- Implement abstract method calculate_performance()
- Show multiple inheritance with class TeachingAssistant

Code:

```
from abc import ABC, abstractmethod
class Person(ABC):
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def __str__(self):
        return f"Name: {self.name}, Age: {self.age}"

    @abstractmethod
    def calculate_performance(self):
        pass


class Student(Person):
    __students_count = 0
    __students_roll_numbers = set()
    def __init__(self, name, age, roll_number, course):
        super().__init__(name, age)
        self.__roll_number = roll_number
        self.course = course
        Student.__students_count += 1
        Student.__students_roll_numbers.add(roll_number)

    @property
    def roll_number(self):
        return self.__roll_number

    @roll_number.setter
    def roll_number(self, value):
        if isinstance(value, int) and value > 0 and value not in
Student.__students_roll_numbers:

Student.__students_roll_numbers.discard(self.__roll_number)
            self.__roll_number = value
            Student.__students_roll_numbers.add(value)
        else:
            raise ValueError("Roll number must be a positive integer
and unique")
```

```
    def __str__(self):
        return super().__str__() + f", Roll Number: {self.roll_number}, Course: {self.course}"

    @property
    def grade(self):
        return self._grade

    @grade.setter
    def grade(self, value):
        if value in ['A', 'B', 'C', 'D', 'F']:
            self._grade = value
        else:
            raise ValueError("Grade must be one of A, B, C, D, F")

    def calculate_performance(self):
        if hasattr(self, '_grade'):
            return f"Student {self.name} has a grade of {self._grade}"
        else:
            return f"Student {self.name} does not have a grade assigned yet"


class Faculty(Person):
    __faculty_count = 0
    __employee_ids = set()
    def __init__(self, name, age, employee_id, department):
        super().__init__(name, age)
        self.__employee_id = employee_id
        self.department = department
        Faculty.__faculty_count += 1
        Faculty.__employee_ids.add(employee_id)

    @property
    def employee_id(self):
        return self.__employee_id

    @employee_id.setter
    def employee_id(self, value):
        if isinstance(value, int) and value > 0 and value not in Faculty.__employee_ids:
            Faculty.__employee_ids.discard(self.__employee_id)
            self.__employee_id = value
            Faculty.__employee_ids.add(value)
        else:
            raise ValueError("Employee ID must be a positive integer")

    @property
    def performance(self):
```

```
        return self.performance

    @performance.setter
    def performance(self, value):
        if isinstance(value, (int, float)) and value >= 0:
            self._performance = value
        else:
            raise ValueError("Performance must be a non-negative
number")

    def __str__(self):
        return super().__str__() + f", Employee ID:
{self.employee_id}, Department: {self.department}"

    def calculate_performance(self):
        if hasattr(self, '_performance'):
            return f"Faculty {self.name} has a performance score of
{self._performance}"
        else:
            return f"Faculty {self.name} does not have a performance
score assigned yet"

print("Creating Student:")
student1 = Student("Vatsal", 21, 102, "Computer Engineering")
print(student1)
student1.roll_number = 142  # Changing roll number
print("After updating roll number:")
print(student1)
print("Creating another Student instance with duplicate roll
number:")
student2 = Student("Honey", 22, 100, "Information Technology")
print(student2)
try:
    print("Attempting to set duplicate roll number for student2:")
    student2.roll_number = 142
except ValueError as e:
    print(e)
print(student2)

print("\nCreating Faculty:")
faculty1 = Faculty("Mr. Karan", 45, 1001, "Computer Science")
print(faculty1)
faculty1.employee_id = 1002  # Changing employee ID
print("After updating employee ID:")
print(faculty1)

print("\nCalculating performance for student1:")
print(student1.calculate_performance())
student1.grade = 'A'  # Assigning grade to student1
print(student1.calculate_performance())
```

```
print("\nCalculating performance for faculty1:")
print(faculty1.calculate_performance())
faculty1.performance = 4.5  # Assigning performance score to faculty1
print(faculty1.calculate_performance())

class TeachingAssistant(Student, Faculty):
    def __init__(self, name, age, roll_number, course, employee_id,
department):
        # Initialize Person attributes first
        Person.__init__(self, name, age)

        # Manually set Student attributes
        self._Student__roll_number = roll_number
        self.course = course
        Student._Student__students_count += 1
        Student._Student__students_roll_numbers.add(roll_number)

        # Manually set Faculty attributes
        self._Faculty__employee_id = employee_id
        self.department = department
        Faculty._Faculty__faculty_count += 1
        Faculty._Faculty__employee_ids.add(employee_id)

    def __str__(self):
        return f"Name: {self.name}, Age: {self.age}, Roll Number:
{self.roll_number},    Course:    {self.course},    Employee    ID:
{self.employee_id}, Department: {self.department}"

    def calculate_performance(self):
        student_performance = Student.calculate_performance(self)
        faculty_performance = Faculty.calculate_performance(self)
        return f"{student_performance}\n{faculty_performance}"

print("\nCreating Teaching Assistant:")
ta1 = TeachingAssistant("Aditi", 23, 105, "Computer Engineering",
1004, "Computer Science")
print(ta1)
ta1.grade = 'B'
ta1.performance = 4.0
print("\nCalculating performance for Teaching Assistant:")
print(ta1.calculate_performance())
```

Output:

```
Creating Student:

Name: Vatsal, Age: 21, Roll Number: 102, Course: Computer

Engineering

After updating roll number:
```

```
Name: Vatsal, Age: 21, Roll Number: 142, Course: Computer
Engineering
Creating another Student instance with duplicate roll number:
Name: Honey, Age: 22, Roll Number: 100, Course: Information
Technology
Attempting to set duplicate roll number for student2:
Roll number must be a positive integer and unique
Name: Honey, Age: 22, Roll Number: 100, Course: Information
Technology

Creating Faculty:
Name: Mr. Karan, Age: 45, Employee ID: 1001, Department: Computer
Science
After updating employee ID:
Name: Mr. Karan, Age: 45, Employee ID: 1002, Department: Computer
Science

Calculating performance for student1:
Student Vatsal does not have a grade assigned yet
Student Vatsal has a grade of A

Calculating performance for faculty1:
Faculty Mr. Karan does not have a performance score assigned yet
Faculty Mr. Karan has a performance score of 4.5

Creating Teaching Assistant:
Name: Aditi, Age: 23, Roll Number: 105, Course: Computer
Engineering, Employee ID: 1004, Department: Computer Science

Calculating performance for Teaching Assistant:
Student Aditi has a grade of B
Faculty Aditi has a performance score of 4.0
```

4. Develop a Shape Calculator:

- Abstract base class Shape with abstract methods area() and perimeter()
- Child classes: Circle, Rectangle, Triangle
- Operator overloading for comparing shapes based on area
- Implement str and repr methods
- Use class methods for creating shapes from different inputs

Code:

```
from abc import ABC, abstractmethod
class Shape(ABC):
    @abstractmethod
    def area(self):
        pass

    @abstractmethod
    def perimeter(self):
        pass

class Circle(Shape):
    def __init__(self, radius):
        self.radius = radius

    def area(self):
        return 3.14159 * self.radius ** 2

    def perimeter(self):
        return 2 * 3.14159 * self.radius

    def set_radius(self, radius):
        self.radius = radius

    def __str__(self):
        return f"Circle with radius: {self.radius}"

    def __repr__(self):
        return f"Circle(radius={self.radius})"


class Rectangle(Shape):
    def __init__(self, width, height):
        self.width = width
        self.height = height

    def area(self):
        return self.width * self.height

    def perimeter(self):
        return 2 * (self.width + self.height)
```

```
    def set_dimensions(self, width, height):
        self.width = width
        self.height = height

    def __str__(self):
        return f"Rectangle with width: {self.width} and height:
{self.height}"

    def __repr__(self):
        return                        f"Rectangle(width={self.width},
height={self.height})"

class Triangle(Shape):
    def __init__(self, side1, side2, side3):
        self.side1 = side1
        self.side2 = side2
        self.side3 = side3

    def area(self):
        s = (self.side1 + self.side2 + self.side3) / 2
        return (s * (s - self.side1) * (s - self.side2) * (s -
self.side3)) ** 0.5

    def perimeter(self):
        return self.side1 + self.side2 + self.side3

    def set_sides(self, side1, side2, side3):
        self.side1 = side1
        self.side2 = side2
        self.side3 = side3

    def __str__(self):
        return f"Triangle with sides: {self.side1}, {self.side2},
{self.side3}"

    def __repr__(self):
        return f"Triangle(side1={self.side1}, side2={self.side2},
side3={self.side3})"

rect = Rectangle(4, 5)
print(f"\n{rect}")
print(repr(rect))
print("Area of rectangle:", rect.area())
print("Perimeter of rectangle:", rect.perimeter())


circle = Circle(3)
print(f"\n{circle}")
print(repr(circle))
print("Area of circle:", circle.area())
```

```
print("Perimeter of circle:", circle.perimeter())

triangle = Triangle(3, 4, 5)
print(f"\n{triangle}")
print(repr(triangle))
print("Area of triangle:", triangle.area())
print("Perimeter of triangle:", triangle.perimeter())
```

Output:

```
Rectangle with width: 4 and height: 5
Rectangle(width=4, height=5)
Area of rectangle: 20
Perimeter of rectangle: 18

Circle with radius: 3
Circle(radius=3)
Area of circle: 28.27431
Perimeter of circle: 18.849539999999998

Triangle with sides: 3, 4, 5
Triangle(side1=3, side2=4, side3=5)
Area of triangle: 6.0
Perimeter of triangle: 12
```

5. Create a Library Management System:

- Class Book with private attributes (__title, __author, __isbn)
- Class Library to manage collection of Book objects
- Implement iterator protocol in Library class
- Use class decorators for logging
- Demonstrate attribute access using getattr, setattr, hasattr
- Example: library.add_book(book1), library.remove_book('ISBN123')

Code:

```
class Book:
    def __init__(self, title, author, isbn):
        self.__title = title
        self.__author = author
        self.__isbn = isbn

    def __str__(self):
        return      f"'{self.__title}'      by      {self.__author}
({self.__isbn})"

    def __repr__(self):
        return                          f"Book(title='{self.__title}',
author='{self.__author}', isbn={self.__isbn})"


class Library:
    def __init__(self):
        self.books = []

    def log_book_addition(func):
        def wrapper(self, book):
            print(f"Adding book: {book}")
            func(self, book)
            print(f"Book added successfully: {book}")
        return wrapper

    def log_book_removal(func):
        def wrapper(self, isbn):
            print(f"Removing book with ISBN: {isbn}")
            func(self, isbn)
            print(f"Book removed successfully with ISBN: {isbn}")
        return wrapper

    @log_book_addition
    def add_book(self, book):
        self.books.append(book)

    def display_books(self):
```

```
        for book in self.books:
            print(book)

    @log_book_removal
    def remove_book(self, isbn):
        self.books = [book for book in self.books if book._Book__isbn
!= isbn]


library = Library()
book1 = Book("The Great Gatsby", "F. Scott Fitzgerald", 1234567890)
book2 = Book("To Kill a Mockingbird", "Harper Lee", 2345678901)
library.add_book(book1)
library.add_book(book2)
print("\nBooks in the library:")
library.display_books()
print("\n")
library.remove_book(1234567890)
print("\nBooks in the library after removal:")
library.display_books()
```

Output:

```
Adding book: 'The Great Gatsby' by F. Scott Fitzgerald (1234567890)
Book added successfully: 'The Great Gatsby' by F. Scott Fitzgerald
(1234567890)
Adding book: 'To Kill a Mockingbird' by Harper Lee (2345678901)
Book added successfully: 'To Kill a Mockingbird' by Harper Lee
(2345678901)

Books in the library:
'The Great Gatsby' by F. Scott Fitzgerald (1234567890)
'To Kill a Mockingbird' by Harper Lee (2345678901)


Removing book with ISBN: 1234567890
Book removed successfully with ISBN: 1234567890

Books in the library after removal:
'To Kill a Mockingbird' by Harper Lee (2345678901)
```